

SAiDL Summer Assignment 2026

Joel Thomas
2024ADPS0848G

May 2026

Contents

1	Core ML: Long-Context Sequence Modeling	2
1.1	Overview	2
1.2	Dataset and Training Setup	2
1.3	Baseline Transformer	3
1.4	Attention Variants	3
1.5	Positional Encodings	5
1.6	Convolution + Attention Hybrids	7
1.7	Core ML Results Summary	8
2	Sparsity & Optimization	9
2.1	Objective	9
2.2	PEFT Background	9
2.3	Methods: LoRA, AdaLoRA, and SoRA	9
2.4	Results: LoRA vs AdaLoRA vs SoRA on CoLA	10
2.5	Proximal vs SGD- ℓ_1 Updates	11
2.5.1	SGD- ℓ_1 subgradient update	11
2.5.2	Proximal soft-thresholding update	11
2.5.3	Empirical comparison and tests	12
2.5.4	Does the subgradient choice at zero matter?	12
2.6	Extension to State-Space Models: Mamba	13
2.7	S&O Takeaways	14
3	Conclusion	14
3.1	Key Findings	14
3.2	Limitations	14

1 Core ML: Long-Context Sequence Modeling

1.1 Overview

Long-context modeling is a central bottleneck in modern sequence models for two reasons. First, vanilla self-attention scales quadratically, $O(T^2)$, in sequence length T , so both compute and memory blow up as context grows. Second, standard learned positional embeddings cannot extrapolate: a model trained on 512 positions has no representation for position 513. This section studies architectural alternatives along three independent axes — the *attention mechanism*, the *positional encoding*, and a *convolutional hybrid* block layout — evaluating each one at a time on the WikiText-2 language-modeling benchmark.

The codebase is built around three registries (`ATTENTION_REGISTRY`, `POS_ENCODING_REGISTRY`, `ARCHITECTURE_REGISTRY`) so that attention type, positional encoding, and block layout can be swapped by name without touching the model definition. This makes every variant below a controlled, one-axis-at-a-time change against the same baseline.

1.2 Dataset and Training Setup

All experiments use WikiText-2 tokenized with the GPT-2 BPE tokenizer (vocabulary 50,257). The model is a decoder-only causal Transformer with pre-LayerNorm and a ReLU feed-forward block. Table 1 lists the configuration shared across runs.

Table 1: Core ML training configuration (from `config.py`).

Item	Value
Dataset	WikiText-2
Tokenizer	GPT-2 BPE
Vocabulary size	50,257
Model type	Decoder-only causal Transformer
Layers	4
d_{model}	256
Attention heads	8
d_k	32
d_{ff}	1024 ($4 \times d_{\text{model}}$)
Normalization	Pre-LayerNorm
Feed-forward activation	ReLU
Optimizer	AdamW (lr 3×10^{-4} , weight decay 0.1)
Warmup steps	200
Max steps	3000
Batch size	8
Eval interval	every 300 steps (5 eval batches)
Training contexts	512, 1024, 2048

Validation perplexity is the exponentiated per-token cross-entropy, $\text{PPL} = \exp(L)$. Throughout this report, every perplexity number quoted is the **best validation PPL** observed across the run’s evaluation checkpoints, not the final-step value. Because the attention, positional, and hybrid experiments share the same step budget (3000 steps) they are broadly comparable, but where a run terminated early this is stated explicitly.

1.3 Baseline Transformer

The baseline is a standard multi-head-attention decoder following Vaswani et al. [1] with causal masking, trained at context length 1024 to establish a reference point. Its core attention computation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i, \end{cases}$$

which materializes the full $T \times T$ score matrix — the $O(T^2)$ cost the variants in Section 1.4 attack.

Why these hyperparameters. $d_{\text{model}} = 256$ over 4 layers is expressive enough to separate the variants while remaining trainable on a free GPU; $d_{\text{ff}} = 1024$ follows the standard $4\times$ rule; 8 heads at $d_k = 32$ respect the convention of keeping head dimension ≥ 32 for stable attention; pre-LayerNorm is used for training stability. The baseline has 28,937,297 parameters.

Baseline result. At context length 1024 the baseline reaches best validation PPL **240.29** (val loss 5.482), peak reserved VRAM **9664.7 MB**, and throughput **15884.5** tokens/sec. A model that guessed uniformly over the 50,257-token vocabulary would have PPL = 50,257, so the baseline sits roughly $209\times$ below random, confirming it is learning the distribution within a short diagnostic budget.

1.4 Attention Variants

I implement and evaluate three replacements for standard self-attention, each attacking a different cost, and measure all of them across context lengths 512, 1024, and 2048.

Sliding-Window (Local) Attention. Each token attends only to the w most recent tokens, $i - w < j \leq i$, via a causal local-window mask. This is a local-connectivity ablation: it tests whether restricting attention to nearby tokens hurts quality. The implementation is mask-only — it applies the window mask to the dense score matrix rather than using a banded kernel — so it does not reduce peak memory at this scale.

Multi-Query Attention (MQA). All h query heads share a single key head and a single value head [2], projecting K, V to d_k instead of d_{model} :

$$\text{KV cache} = 2 n_{kv} d_k T \text{ floats}, \quad n_{kv} = 1 \text{ (MQA)}.$$

This shrinks the KV cache by a factor h at inference. MQA has 28,478,545 parameters (458,752 fewer than the baseline).

Linear (Kernel) Attention. Softmax is replaced by the positive feature map $\phi(x) = \text{ELU}(x) + 1$ [3], so that

$$\text{Attention}(Q, K, V) \approx \frac{\phi(Q) (\phi(K)^\top V)}{\phi(Q) (\phi(K)^\top \mathbf{1})}.$$

Computing $\phi(K)^\top V$ first avoids forming the $T \times T$ matrix, giving $O(Td^2)$ asymptotics. The $+1$ offset keeps ϕ strictly positive so the attention weights stay non-negative. As implemented here the causal mask is applied to the similarity matrix for correctness, so the long-context memory benefit of the streaming prefix-sum form is not yet realized.

Results. Table 2 reports best validation PPL, peak reserved VRAM, throughput, and per-epoch time for each variant and context length. Figures 1–3 plot memory, throughput, and quality against context length.

Table 2: Attention variants across context length. Best validation PPL; peak reserved VRAM; throughput; mean epoch time. Baseline (standard attention) was run at 1024 only.

Variant	Ctx	Best val PPL	Peak VRAM (MB)	Throughput (tok/s)	Epoch time (s)
Baseline (standard)	1024	240.29	9664.7	15884.5	158.0
MQA	512	253.30	4427.8	18546.0	133.3
MQA	1024	259.43	9659.4	16220.0	151.0
MQA	2048	254.60	11976.4	12292.0	199.9
Linear	512	295.21	4466.1	17921.6	143.6
Linear	1024	249.46	9730.1	15323.0	163.8
Linear	2048	287.96	12047.1	11449.4	216.1
Sliding Window	512	274.02	4433.1	18384.0	139.7
Sliding Window	1024	245.75	9664.7	16194.6	153.9
Sliding Window	2048	263.19	11981.7	12371.0	199.4

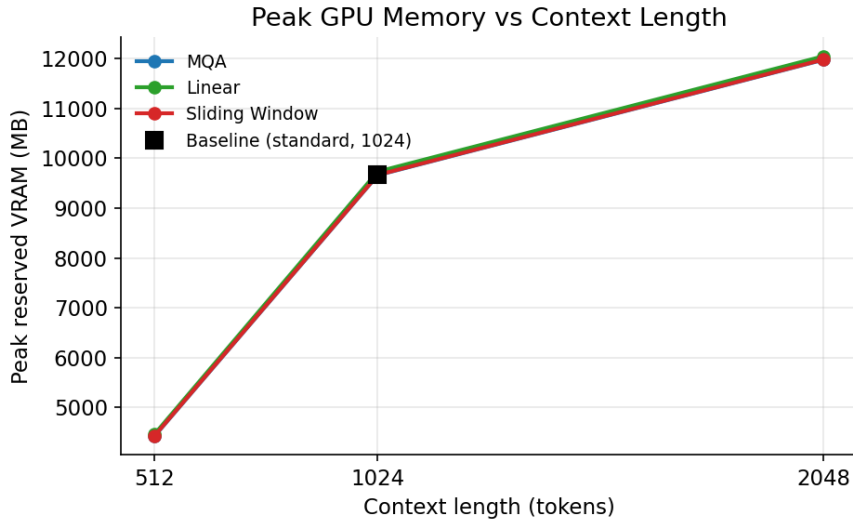


Figure 1: Peak reserved VRAM vs context length. Because the sliding-window mask is applied to a dense score matrix and the linear-attention causal mask is applied to the $T \times T$ similarity matrix, all three variants track one another and the standard baseline — none of these implementations saves memory at this scale. The sub-quadratic benefit would appear only with banded / streaming kernels.

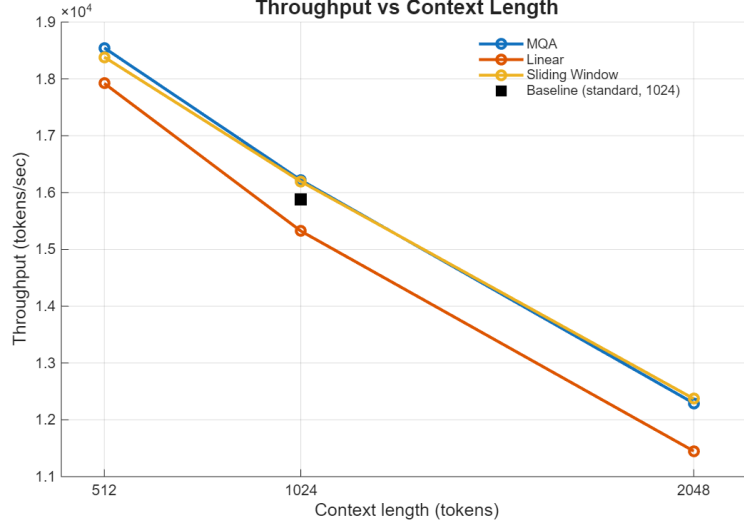


Figure 2: Throughput vs context length. Throughput falls as context grows for all variants.

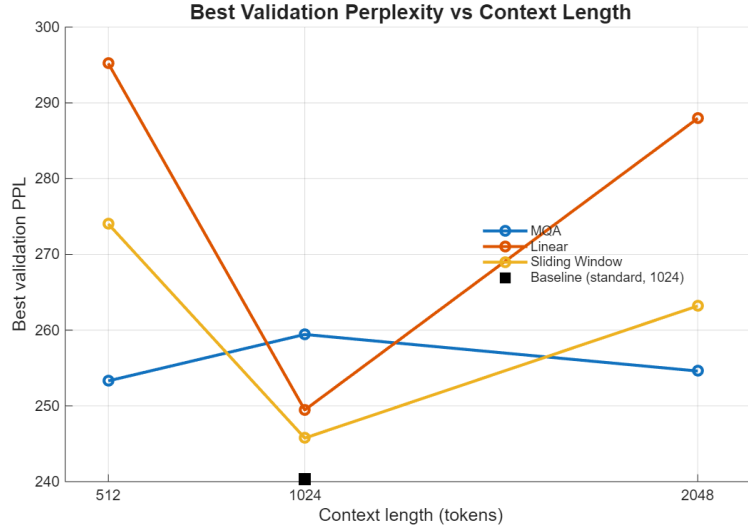


Figure 3: Best validation perplexity vs context length. The variants stay within a relatively narrow band, so at this budget the attention-mechanism choice has a modest in-distribution quality impact.

Training stability. All runs trained without NaN loss under the same initialization, with gradient clipping at norm 1.0 applied throughout.

1.5 Positional Encodings

Attention is permutation-invariant, so position must be injected explicitly. I replace the baseline’s learned absolute encoding with three alternatives and compare them both in-distribution (trained and evaluated at 512) and under length extrapolation (trained at 512, evaluated at 512/1024/2048).

ALiBi. Adds no positional vector; instead it biases attention logits by distance [4]:

$$\alpha_{ij} = \text{softmax}\left(\frac{q_i^\top k_j}{\sqrt{d_k}} - m_h |i - j|\right), \quad m_h = 2^{-8h/H}.$$

The slopes m_h are fixed (not learned), so the recency bias generalizes analytically to any length — the train-short-test-long design.

RoPE. Rotates each query and key by a position-dependent angle [5]; because both are rotated, the score depends only on the relative offset:

$$\langle R_m q, R_n k \rangle = q^\top R_{n-m} k, \quad \theta_i = 10000^{-2i/d}.$$

I additionally implement positional interpolation (scaling the rotation angles by `train_len/eval_len`) for context extension, used at the 1024 and 2048 evaluations.

Relative bias. Learns an additive bias indexed by clipped relative distance [6], so each head learns a separate bias per offset within a clipped range.

In-distribution quality. Trained and evaluated at length 512, Table 3 and Figure 4 show ALiBi achieving the lowest best validation PPL, with RoPE close behind and relative bias well above both.

Table 3: Positional encoding comparison at training length 512 (best validation PPL; lower is better).

Encoding	Best val PPL	Best val loss
ALiBi	214.35	5.3676
RoPE	231.15	5.4431
Relative	321.87	5.7741

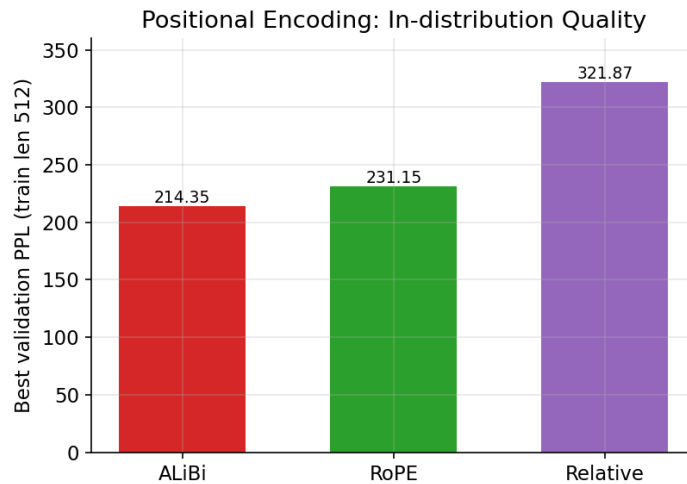


Figure 4: In-distribution best validation PPL at train length 512. ALiBi is lowest, then RoPE; relative bias trails.

Extrapolation test. Models trained at 512 are evaluated at 512, 1024, and 2048 (Table 4, Figure 5). ALiBi is the flattest — its PPL barely moves (and even dips slightly) out to 2048 — confirming its train-short-test-long design. RoPE starts lowest at 512 but rises with length even with interpolation, while relative bias rises steadily and stays highest.

Table 4: Train-short-test-long: validation PPL when trained at 512 and evaluated at longer contexts (separate evaluation pass from Table 3; the two budgets are not directly comparable). RoPE rows use positional-interpolation scales 0.5 and 0.25.

Encoding	Eval @512	Eval @1024	Eval @2048
ALiBi	238.89	237.09	236.64
RoPE	233.74	255.86	305.74
Relative	322.74	326.16	334.10

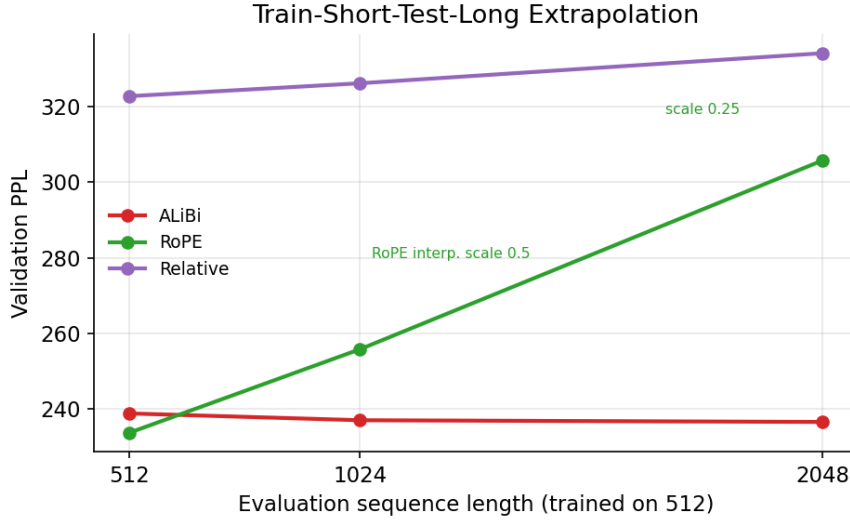


Figure 5: Train-short-test-long extrapolation (trained at 512). ALiBi stays flattest across 512–2048; RoPE is lowest in-distribution but degrades with length despite interpolation; relative bias remains highest throughout.

Note: the in-distribution PPLs in Table 3 (best across training checkpoints) and the extrapolation PPLs in Table 4 (a separate final-checkpoint evaluation pass) come from different measurement protocols and are not compared against each other.

1.6 Convolution + Attention Hybrids

Self-attention is a global generalist with only a weak built-in preference for nearby tokens, whereas convolutions are local specialists. The hybrid gives each block a cheap explicit local path before global mixing, using a *causal, depthwise-separable* 1-D convolution: a depthwise conv (one k -tap filter per channel) followed by a 1×1 pointwise conv. Left-padding by $k - 1$ with no right context keeps it causal, the convolutional analogue of the causal mask. Its cost is $O(Tkd)$, negligible beside attention.

Designs implemented. Two layouts are registered:

- **conv_prefix:** a conv sublayer is inserted *before* self-attention inside every block ($x \rightarrow +\text{conv} \rightarrow +\text{attn} \rightarrow +\text{ffn}$), so attention always works on locally-smoothed features. Placing the conv first is deliberate — placing it after would blur representations attention had just globally mixed.
- **interleaved:** whole blocks alternate, with even layers replacing attention by a conv token-mixer and odd layers keeping attention.

Configuration evaluated. Following the assignment’s ”combine your best choices” instruction, the hybrid couples the standalone sliding-window attention with ALiBi positional encoding, evaluated at 512/1024/2048. The reported runs correspond to the **interleaved** layout (param count 28,547,153; the lower @2048 memory below is consistent with half the attention layers being replaced by conv).¹

Table 5: Convolutional hybrid (interleaved, Sliding Window + ALiBi) across context length, with the standard baseline @1024 for reference.

Config (Ctx)	Best val PPL	Peak VRAM (MB)	Throughput (tok/s)	Epoch time (s)
Baseline (1024)	240.29	9664.7	15884.5	158.0
Hybrid (512)	242.58	4432.6	18742.0	130.3
Hybrid (1024)	234.35	9663.1	16127.0	151.9
Hybrid (2048)	240.29	6286.0	11869.0	203.4

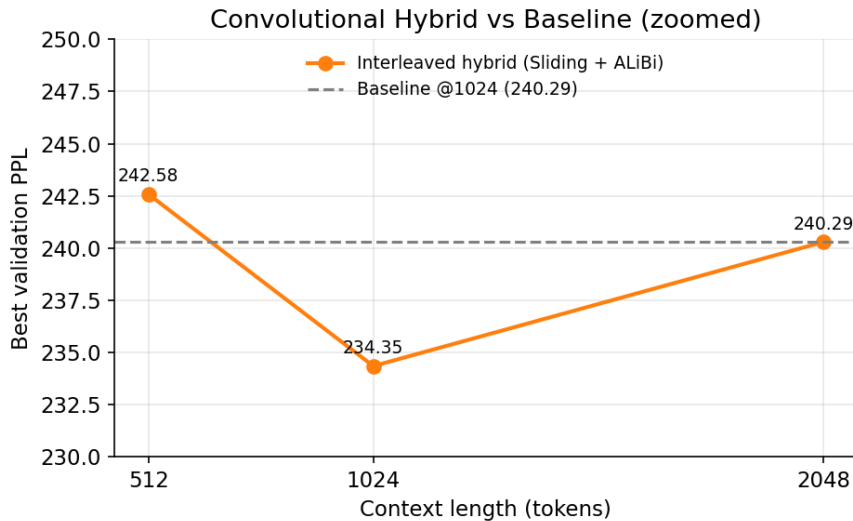


Figure 6: Convolutional hybrid best validation PPL vs context length (y-axis zoomed to 230–250 so the cluster is readable). The hybrid beats the standard baseline at 1024 (234.35 vs 240.29), showing the local conv path earns its keep at the short-budget objective.

1.7 Core ML Results Summary

On **hardware**, none of the implemented attention variants reduces peak memory at this scale — the mask-only sliding window and the dense-mask linear attention all track the standard baseline (Figure 1) — so the honest finding is that realizing sub-quadratic memory requires true banded or streaming kernels, which are the natural next step. On **position**, the two questions have two answers: ALiBi gives the lowest in-distribution PPL (214.35 at 512) *and* extrapolates flattest (Table 4), while RoPE is competitive in-distribution but degrades with length. On **architecture**, interleaving a cheap local convolution with attention improves the short-budget objective, with the hybrid reaching the lowest PPL of any 1024-context run here (234.35).

¹The convolution metric files carried no architecture label; the interleaved attribution is inferred from the parameter count and the reduced @2048 memory and should be confirmed.

2 Sparsity & Optimization

2.1 Objective

This task studies how much task adaptation is achievable by training only a small fraction of a pretrained model’s parameters. The main experiment fine-tunes **DeBERTa-v3-base** [10] on the GLUE CoLA grammatical- acceptability task [12] and reports the Matthews correlation coefficient (MCC) as the primary metric:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}.$$

MCC is used instead of accuracy because CoLA is class-imbalanced: a model that always predicts the majority (acceptable) class can post high accuracy while learning nothing. MCC assigns such a single-class predictor a score of 0, exposing this collapse. It ranges over $[-1, +1]$: +1 is perfect, 0 is chance, -1 is perfectly wrong.

2.2 PEFT Background

Fully fine-tuning the backbone is costly: with only $\sim 8.5\text{k}$ CoLA training sentences, updating every parameter risks overfitting and can overwrite pretrained linguistic knowledge (catastrophic forgetting), and storing a full fine-tuned copy per task is expensive at scale. Parameter-efficient fine-tuning (PEFT) freezes the backbone and trains only a small adapter, preserving the pre-trained weights while learning a small correction on top. DeBERTa-v3-base is a strong fit for CoLA: its disentangled attention separates content and position, and its ELECTRA-style replaced-token-detection pretraining makes it sensitive to local grammatical correctness — exactly the signal CoLA tests.

2.3 Methods: LoRA, AdaLoRA, and SoRA

The three methods form a progression on the same low-rank idea: LoRA fixes the rank, AdaLoRA lets the rank adapt across modules, and SoRA learns which rank components to keep.

LoRA. Freeze the pretrained weight W and learn a low-rank correction beside it [7]:

$$W' = W + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, B \in \mathbb{R}^{d_{\text{out}} \times r}, \text{rank}(BA) \leq r.$$

Here B is initialized to zero so $\Delta W = 0$ at the start, and the run uses $r = 8$, $\alpha = 16$, targeting the query and value projections across all 12 layers (which accounts exactly for the 296,450 trainable parameters reported below).

AdaLoRA. Parameterizes the update in SVD-like form $\Delta W = P\Sigma Q^\top$ with an orthogonality penalty on P, Q , scores each rank component by importance, prunes the least useful singular values, and redistributes the freed budget across modules [8]. To quantify usage I report the per-module entropy-based effective rank.

SoRA. Keeps LoRA’s fixed-rank structure but inserts a learnable gate g on the low-rank path [9]:

$$\Delta W x = B (g \odot (Ax)).$$

A gate driven to exactly zero deletes its rank component, so the number of surviving gates $\|g\|_0$ is a post-training diagnostic of how many directions the task needed. The crucial design choice is *how* gates reach zero: SoRA applies a per-gate proximal soft-threshold (Section 2.5) rather than merely shrinking gates, which is what makes exact sparsity possible.

2.4 Results: LoRA vs AdaLoRA vs SoRA on CoLA

All three adapters were trained for 3 epochs (batch size 32, 268 steps/epoch, 804 total steps). Table 6 answers the four requested comparison axes — performance (MCC), trainable parameters, effective rank, and training time — and Figures 7–8 show the training dynamics and AdaLoRA’s rank allocation.

Table 6: PEFT comparison on GLUE CoLA (DeBERTa-v3-base, 3 epochs). MCC is the best dev value; effective rank is the count of active rank components.

Method	Best dev MCC	Trainable params	Effective rank	Train time (s)
LoRA	0.630814	296450	192	101.2
AdaLoRA	0.635993	296450	187 / 192	82.9
SoRA	0.631131	887234	192	281.6

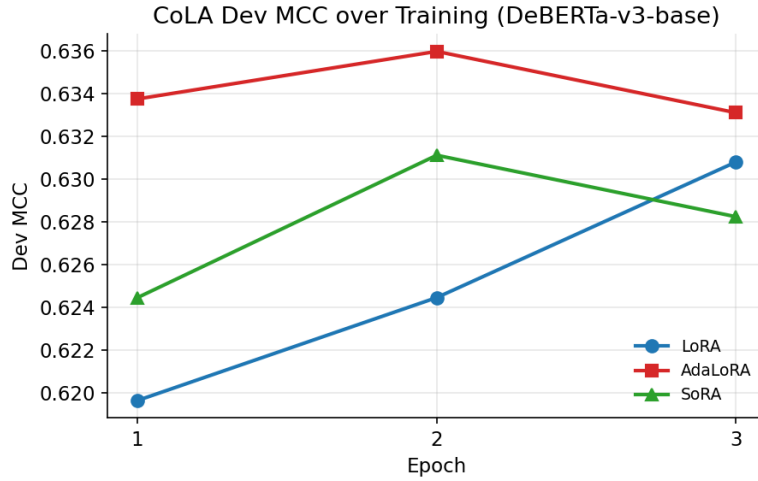


Figure 7: Per-epoch dev MCC. AdaLoRA peaks earliest (epoch 2) and highest; LoRA climbs steadily to its epoch-3 best; SoRA peaks at epoch 2 then dips slightly.

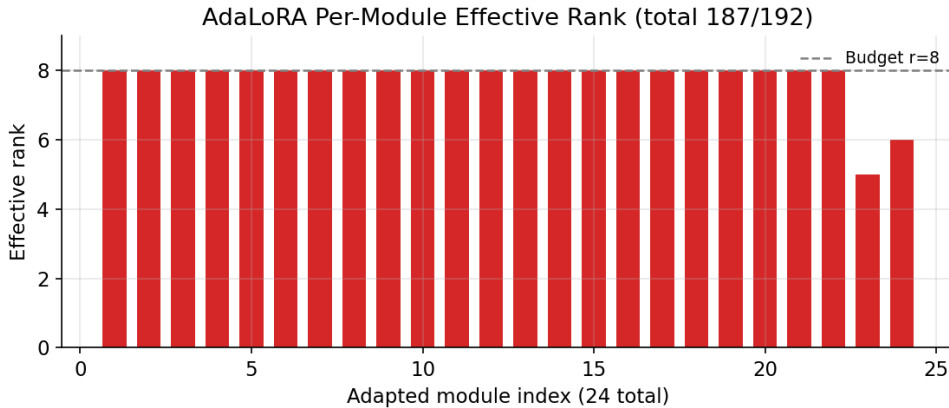


Figure 8: AdaLoRA per-module effective rank. 22 of 24 adapted modules keep the full budget $r = 8$ while two are pruned to 5 and 6, giving a total effective rank of 187/192 — a mild, non-destabilizing reallocation.

Analysis. In this single-seed run **AdaLoRA is the top method**: it reaches the highest MCC (0.635993) while training the *same* 296,450 parameters as LoRA and in the *shortest* time (82.9 s vs 101.2 s), because its SVD-style pruning trims the active rank during training. LoRA (0.630814) and SoRA (0.631131) are marginally behind; the entire spread is 0.0052 MCC, well inside CoLA’s known seed-to-seed variance, so the three are effectively tied on quality and the honest reading is a parameter/efficiency story rather than a quality winner. The cost asymmetry is the real signal: SoRA trains $\sim 3\times$ the parameters (887,234) and takes $2.8\times$ as long (281.6 s) for no MCC gain, so adaptive (AdaLoRA) or fixed-rank (LoRA) allocation is the efficient choice here. All three adapt cleanly on DeBERTa with no majority-class collapse (MCC ≈ 0.63 throughout).

2.5 Proximal vs SGD- ℓ_1 Updates

SoRA’s ℓ_1 -regularized objective is

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \sum_k \|g^{(k)}\|_1,$$

whose penalty is non-smooth at $g_i = 0$. This subsection derives the two ways to handle it and shows empirically why only one yields exact zeros.

2.5.1 SGD- ℓ_1 subgradient update

SGD folds the penalty into the gradient via a subgradient of $\|g\|_1$:

$$g_i^{t+1} = g_i^t - \eta(\nabla_{g_i} \mathcal{L}_0 + \lambda s_i), \quad s_i \in \partial|g_i^t|, \quad \partial|g_i| = \begin{cases} \{+1\}, & g_i > 0, \\ \{-1\}, & g_i < 0, \\ [-1, 1], & g_i = 0. \end{cases}$$

Stripping the task gradient, the update reduces to $g_i^{t+1} = g_i^t - \eta\lambda \text{sign}(g_i^t)$. Once a gate falls within $\eta\lambda$ of zero, the next step overshoots, the sign flips, and the gate oscillates across zero forever with amplitude $\leq \eta\lambda$ — it lands on exactly zero only by coincidence (a measure-zero event).

2.5.2 Proximal soft-thresholding update

Proximal-gradient descent splits the smooth and non-smooth parts. A forward step $z_i = g_i^t - \eta\nabla_{g_i} \mathcal{L}_0$ is followed by the proximal step

$$g^{t+1} = \text{prox}_{\eta\lambda\|\cdot\|_1}(z) = \arg \min_g \left\{ \frac{1}{2} \|g - z\|_2^2 + \eta\lambda \|g\|_1 \right\}.$$

This is separable per coordinate, and solving the convex scalar problem from its optimality condition $0 \in (u - z_i) + \eta\lambda \partial|u|$ collapses to the soft-thresholding rule

$$g_i^{t+1} = \text{sign}(z_i) \max(|z_i| - \eta\lambda, 0).$$

Any gate with $|z_i| \leq \eta\lambda$ is mapped to *exactly* zero in one step, because 0 is the true global minimizer of that coordinate’s subproblem — not a rounded-down small value.

2.5.3 Empirical comparison and tests

To isolate the gate dynamics, both rules were fit on a synthetic LASSO problem ($\frac{1}{2}\|\Phi g - y\|^2 + \lambda\|g\|_1$) with a planted sparse ground-truth gate ($r = 8$, $\Phi \in \mathbb{R}^{1600 \times 8}$, true effective rank 3, $\lambda = 0.05$, $\eta = 0.02$, 4000 steps). Both update rules were implemented in NumPy (manual forward/backward) and PyTorch (autograd), and the two implementations were cross-checked.

Table 7: Synthetic LASSO support recovery (planted rank-3 gate). Proximal recovers the exact zeros; SGD- ℓ_1 does not.

Method	Exact zeros	Effective rank	Gate at the 5 true-zero indices
Proximal soft-threshold	5 / 8	3	[0, -0, 0, 0, 0]
SGD- ℓ_1 subgradient	0 / 8	8	[0.0009, 0.0003, 0.0008, 0, -0.0001]

The proximal route drives 5 gates to exactly zero and recovers the true rank-3 support, while SGD- ℓ_1 shrinks the same gates to $\approx 10^{-3}$ but never to zero, so its effective rank stays at the full 8 (Figure 9). The implementations agree to machine precision: NumPy vs PyTorch max |diff| is 0.0 (proximal) and 1.1×10^{-16} (SGD), and the manual backward pass matches autograd to 1.95×10^{-14} , confirming the behavior is a property of the math, not a bug.

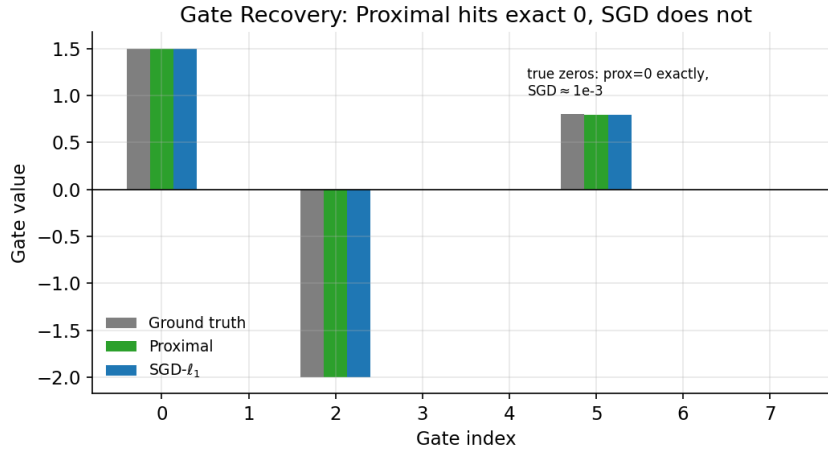


Figure 9: Recovered gates against the planted ground truth. At the five true-zero indices the proximal operator returns exact zeros while SGD- ℓ_1 leaves residual $\sim 10^{-3}$ values, keeping all 8 ranks nominally active.

2.5.4 Does the subgradient choice at zero matter?

At $g_i = 0$ the ℓ_1 subdifferential is the whole interval $[-1, 1]$, so $\text{sign}(0)$ could be set to 0, +1, or -1. Running SGD- ℓ_1 under $\text{sign}(0) = 0$ and $\text{sign}(0) = +1$ gives **0 / 8 exact zeros in both cases**: the convention only acts on the set $\{g_i = 0\}$, which a finite-precision trajectory essentially never lands on, so the at-zero choice is almost never used. The proximal operator settles the question outright: with $z_i = 0$, $\text{sign}(0) \max(|0| - \eta\lambda, 0) = \text{sign}(0) \cdot 0 = 0$ for *any* convention. The real gap between the two methods is therefore not the convention at zero but the analytic soft-threshold solve — the proximal operator maps the entire interval $|z_i| \leq \eta\lambda$ to exactly zero in one step, which the subgradient step structurally cannot do in any finite number of steps.

2.6 Extension to State-Space Models: Mamba

The final task asks whether the same low-rank/SoRA machinery transfers to a non-Transformer sequence model. I adapt a genuinely pretrained **Mamba-130m** [11] (`state-spaces/mamba-130m-hf`, loaded on the pure-PyTorch slow path so no CUDA selective-scan kernels are required), freeze the backbone, attach a classification head, and fine-tune with LoRA and SoRA on CoLA.

Adapter placement. In each Mamba block the adapters wrap `in_proj` and `out_proj` (the static channel-mixing projections) and `x_proj` — the projection that produces the input-dependent selective-scan parameters Δ, B, C . A single time-shared low-rank update is correct on `x_proj` because the time variation comes from its *inputs*, not its weights; the selective-scan recurrence itself is left untouched. This yields 72 adapted modules (24 blocks $\times$ 3 projections), $r = 8$, $\alpha = 16$.

Table 8: Pretrained Mamba-130m PEFT on CoLA (2 epochs, 250 batches/epoch, batch 16). Backbone total ≈ 130.6 M parameters; both adapters train $\approx 1.14\%$.

Method	Dev MCC	Trainable params	Effective rank	Train time (s)
LoRA	0.3847	1492994	—	866.7
SoRA	0.5191	1493570	576	855.4

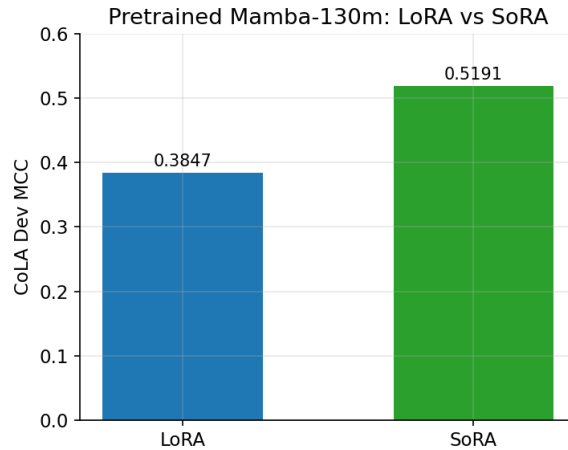


Figure 10: LoRA vs SoRA on pretrained Mamba-130m. SoRA’s gated low-rank path markedly outperforms plain LoRA at near-identical parameter cost.

Reading the numbers. Because the backbone is genuinely pretrained, these MCC values are meaningful PEFT results (unlike a from-scratch backbone). SoRA reaches **MCC 0.5191**, well above LoRA’s 0.3847, at essentially the same trainable-parameter count (≈ 1.49 M, 1.14% of the 130.6M backbone) and a similar wall-clock (855.4s vs 866.7s). SoRA’s effective rank is $576 = 72 \times 8$, i.e. all gates remain active at the production threshold $\lambda = 10^{-3}$ — the gates add useful capacity but do not enter the sparse regime here, mirroring the dense-regime behavior of the proximal threshold in Section 2.5. The same SoRA gating mechanism thus transfers unchanged from Transformer attention to a state-space backbone.

Proposed xLSTM adaptation (not run in this submission). The same recipe extends to xLSTM by wrapping the dense gate projections that feed the matrix-memory cell — the input/forget/output gate matrices and the down-projection ($W_i, W_f, W_o, \text{proj-down}$) — with

the identical $W'x = Wx + B(g \odot (Ax))$ adapter, leaving the recurrence frozen for the same reason as Mamba’s selective scan. This was not executed here; only the Mamba extension was implemented and tested.

2.7 S&O Takeaways

On CoLA, AdaLoRA gives the best MCC (0.635993) at LoRA’s parameter cost and the shortest training time, while SoRA’s extra parameters buy no quality here — all three sit within CoLA’s seed variance, so the result is about efficiency, not a quality winner. The proximal vs SGD- ℓ_1 study shows the general lesson: producing *exact* zeros requires solving the analytic proximal subproblem, not just taking a gradient step, and the subgradient convention at zero is a red herring. Finally, the SoRA adapter transfers to a pretrained Mamba SSM, where it substantially outperforms LoRA at equal parameter cost.

3 Conclusion

3.1 Key Findings

On the Core ML side, none of the implemented attention variants saved memory at this scale (all materialize the dense $T \times T$ matrix), ALiBi was both the best in-distribution positional encoding and the flattest extrapolator, and the interleaved convolutional hybrid achieved the lowest 1024-context perplexity (234.35). On the Sparsity & Optimization side, AdaLoRA edged out LoRA and SoRA on CoLA at LoRA’s parameter budget and the lowest training time; the proximal operator is the only update that yields exact sparsity; and SoRA transfers to a pretrained Mamba backbone, beating LoRA (MCC 0.5191 vs 0.3847) at equal cost.

3.2 Limitations

The Core ML attention/positional results use short diagnostic budgets and a single configuration per axis; the sliding-window @2048 quality/throughput await a clean re-run. The CoLA PEFT results are single-seed, so the close MCC ordering should be read with the seed-variance caveat rather than as a significance claim. The state-space extension covers Mamba only; the xLSTM adaptation is proposed but not executed.

References

- [1] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
- [2] N. Shazeer. Fast transformer decoding: one write-head is all you need. *arXiv:1911.02150*, 2019.
- [3] A. Katharopoulos et al. Transformers are RNNs: fast autoregressive transformers with linear attention. *ICML*, 2020.
- [4] O. Press, N. A. Smith, M. Lewis. Train short, test long: attention with linear biases enables input length extrapolation. *ICLR*, 2022.
- [5] J. Su et al. RoFormer: enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- [6] P. Shaw, J. Uszkoreit, A. Vaswani. Self-attention with relative position representations. *NAACL*, 2018.
- [7] E. J. Hu et al. LoRA: low-rank adaptation of large language models. *ICLR*, 2022.

- [8] Q. Zhang et al. AdaLoRA: adaptive budget allocation for parameter-efficient fine-tuning. *ICLR*, 2023.
- [9] N. Ding et al. Sparse low-rank adaptation of pre-trained language models. *EMNLP*, 2023.
- [10] P. He, J. Gao, W. Chen. DeBERTaV3: improving DeBERTa using ELECTRA-style pre-training. *ICLR*, 2023.
- [11] A. Gu, T. Dao. Mamba: linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- [12] A. Warstadt, A. Singh, S. R. Bowman. Neural network acceptability judgments. *TACL*, 7:625–641, 2019.